Semaine 9 — Rendre le robot explorateur jouable

Tutoriel étudiant — Contrôleur caméra et interactions 3D

Cours 420-511-MV — Développement de jeux vidéo
Unity 6000.2.5f1 — Automne 2026

Votre mission

Vous poursuivez le prototype de la Semaine 8. Vous allez programmer le déplacement du robot, sa rotation, une caméra suiveuse, la collecte des batteries, l'ouverture de la porte, une zone dangereuse et un interrupteur activé avec la touche E.

Boucle de jeu obtenue

1. Déplacer le robot dans le niveau.
2. Collecter les trois batteries.
3. Éviter la zone dangereuse.
4. Ouvrir la porte lorsque l'objectif est atteint.
5. Atteindre la sortie.

Fichiers à créer

Script	Rôle
ControleurRobot3D.cs	Déplacement et rotation
CameraSuiveuse3D.cs	Suivi fluide
BatterieCollectable.cs	Détection de collecte
GestionnaireJeu3D.cs	Compteur et progression
PorteSortie3D.cs	Ouverture de la porte
ZoneDangereuse3D.cs	Réapparition
Interrupteur3D.cs	Interaction avec E

Avant de commencer. Dupliquez votre projet ou créez une branche Git. Ouvrez la scène de la Semaine 8 et vérifiez qu'aucune erreur rouge ne se trouve dans la Console.

1 Préparer le robot

6. Sélectionnez Robot_Explorateur.
7. Vérifiez la présence d'un Collider 3D.
8. Vérifiez la présence d'un Rigidbody.
9. Désactivez Is Kinematic.
10. Activez Interpolate.
11. Dans Constraints, figez la rotation X et Z.
12. Créez le script ControleurRobot3D.cs et attachez-le au robot.

Pourquoi figer X et Z

Le robot doit pouvoir tourner autour de Y, mais ne doit pas basculer lorsqu'il touche un obstacle.

2 Lire les commandes

Ajoutez les variables et lisez les axes dans Update :

```
[SerializeField] private float vitesse = 5f;
[SerializeField] private float vitesseRotation = 10f;
private Rigidbody corps;
private Vector3 direction;

private void Awake()
{
   corps = GetComponent<Rigidbody>();
}

private void Update()
{
   float horizontal = Input.GetAxisRaw("Horizontal");
   float vertical = Input.GetAxisRaw("Vertical");
   direction = new Vector3(horizontal, 0f, vertical);
   direction = Vector3.ClampMagnitude(direction, 1f);
}
```

Validation 1. Ajoutez temporairement Debug.Log(direction) et vérifiez les valeurs X et Z. Retirez ensuite le message.

3 Déplacer avec le Rigidbody

Ajoutez FixedUpdate :

```
private void FixedUpdate()
{
   Vector3 deplacement = direction * vitesse * Time.fixedDeltaTime;
   corps.MovePosition(corps.position + deplacement);
}
```

Comprendre la normalisation

Sans limitation, la combinaison de deux axes produit une diagonale plus rapide. ClampMagnitude conserve les petites valeurs et limite la longueur maximale à 1.

Entrée	Vecteur brut	Magnitude approximative
Droite	1, 0, 0	1
Avant	0, 0, 1	1
Diagonale	1, 0, 1	1,41
Diagonale limitée	0,71, 0, 0,71	1

4 Orienter le robot

Dans FixedUpdate, après le déplacement :

```
if (direction.sqrMagnitude > 0.001f)
{
   Quaternion cible = Quaternion.LookRotation(direction, Vector3.up);
   Quaternion rotation = Quaternion.Slerp(
      corps.rotation, cible, vitesseRotation * Time.fixedDeltaTime);
   corps.MoveRotation(rotation);
}
```

Validation 2. Le robot avance sur X et Z, conserve la même vitesse en diagonale et regarde dans la direction du mouvement.

5 Créer la caméra suiveuse

13. Créez CameraSuiveuse3D.cs.
14. Attachez-le à Main Camera.
15. Exposez une référence cible vers le robot.
16. Ajoutez un décalage et une vitesse de suivi.
17. Effectuez le suivi dans LateUpdate.

```
public class CameraSuiveuse3D : MonoBehaviour
{
   [SerializeField] private Transform cible;
   [SerializeField] private Vector3 decalage = new Vector3(0f, 8f, -10f);
   [SerializeField] private float fluidite = 6f;


   private void LateUpdate()
   {
      if (cible == null) return;
      Vector3 positionVoulue = cible.position + decalage;
      transform.position = Vector3.Lerp(
         transform.position, positionVoulue, fluidite * Time.deltaTime);
      transform.LookAt(cible.position + Vector3.up);
   }
}
```

18. Dans l'Inspector, glissez Robot_Explorateur dans Cible.
19. Testez les valeurs du décalage.
20. Vérifiez que la caméra suit après le déplacement du robot.

Question

Pourquoi LateUpdate est-il adapté à une caméra qui suit un objet déplacé pendant la même image?

_____

6 Créer le gestionnaire de jeu

Créez un GameObject vide nommé GESTIONNAIRE_JEU, puis attachez GestionnaireJeu3D.cs :

```
public class GestionnaireJeu3D : MonoBehaviour
{
    [SerializeField] private int objectifBatteries = 3;
    [SerializeField] private PorteSortie3D porte;
    private int batteriesCollectees;

    public void AjouterBatterie()
    {
        batteriesCollectees++;
        Debug.Log($"Batteries : {batteriesCollectees}/{objectifBatteries}");
        if (batteriesCollectees >= objectifBatteries)
            porte.Ouvrir();
    }
}
```

7 Programmer les batteries

Créez BatterieCollectable.cs et ajoutez-le au prefab PF_Batterie :

```
public class BatterieCollectable : MonoBehaviour
{
    [SerializeField] private GestionnaireJeu3D gestionnaire;
    private bool collectee;

    private void OnTriggerEnter(Collider autre)
    {
        if (collectee || !autre.CompareTag("Player")) return;
        collectee = true;
        gestionnaire.AjouterBatterie();
        Destroy(gameObject);
    }
}
```

21. Créez ou attribuez le tag Player au robot.
22. Vérifiez que le Collider de la batterie utilise Is Trigger.
23. Associez le gestionnaire aux instances ou trouvez-le au démarrage selon la méthode demandée.

Validation 3. Chaque batterie disparaît une seule fois et un message correct apparaît dans la Console.

8 Programmer la porte

Attachez PorteSortie3D.cs au parent Porte_Sortie :

```
public class PorteSortie3D : MonoBehaviour
{
   [SerializeField] private Transform battant;
   [SerializeField] private Vector3 decalageOuverture = new Vector3(0f, 3f, 0f);
   [SerializeField] private float vitesse = 2f;
   private Vector3 positionFermee;
   private Vector3 positionOuverte;
   private bool ouverte;

   private void Awake()
   {
      positionFermee = battant.localPosition;
      positionOuverte = positionFermee + decalageOuverture;
   }

   private void Update()
   {
      Vector3 cible = ouverte ? positionOuverte : positionFermee;
      battant.localPosition = Vector3.MoveTowards(
         battant.localPosition, cible, vitesse * Time.deltaTime);
   }

   public void Ouvrir() => ouverte = true;
}
```

24. Glissez Battant dans le champ correspondant.
25. Glissez Porte_Sortie dans le gestionnaire.
26. Testez avant et après la troisième batterie.

Expliquez

Pourquoi le gestionnaire appelle-t-il une méthode publique de la porte au lieu de déplacer directement son Transform?

_____

9 Ajouter une zone dangereuse

27. Créez un Cube aplati et nommez-le Zone_Dangereuse.
28. Donnez-lui un matériau rouge.
29. Cochez Is Trigger sur son Collider.
30. Créez un GameObject Point_Reapparition près du départ.
31. Ajoutez ZoneDangereuse3D.cs à la zone.

```
public class ZoneDangereuse3D : MonoBehaviour
{
   [SerializeField] private Transform pointReapparition;

   private void OnTriggerEnter(Collider autre)
   {
      if (!autre.CompareTag("Player")) return;
      Rigidbody rb = autre.attachedRigidbody;
      if (rb == null) return;
      rb.position = pointReapparition.position;
      rb.linearVelocity = Vector3.zero;
      rb.angularVelocity = Vector3.zero;
   }
}
```

Validation 4. Le robot revient au départ sans conserver une vitesse précédente et la caméra le rejoint correctement.

10 Créer une interaction avec E

L'interrupteur détecte d'abord la proximité. L'action est déclenchée seulement si le robot se trouve dans la zone et que la touche E est pressée.

```
private bool joueurProche;

private void Update()
{
   if (joueurProche && Input.GetKeyDown(KeyCode.E))
      porte.Ouvrir();
}

private void OnTriggerEnter(Collider autre)
{
   if (autre.CompareTag("Player")) joueurProche = true;
}
```

```
private void OnTriggerExit(Collider autre)
{
    if (autre.CompareTag("Player")) joueurProche = false;
}
```

11 Défi autonome

Ajoutez une seconde porte qui respecte les contraintes suivantes :

- elle reste fermée au démarrage;
- elle possède un interrupteur distinct;
- l’interrupteur fonctionne seulement à proximité;
- la touche E déclenche l’action;
- un message dans la Console indique si l’action est permise;
- les scripts ne contiennent pas de référence codée vers un nom de GameObject.

12 Lancer le projet final

32. Formez une équipe de trois ou quatre personnes.
33. Choisissez un concept 3D réalisable.
34. Définissez le joueur, la caméra, l’objectif, le danger et la fin de partie.
35. Créez le dépôt Git et invitez tous les membres.
36. Complétez le document de conception.
37. Attribuez une première tâche technique à chaque membre.
38. Effectuez un premier commit personnel.

Membre	Première responsabilité	Preuve prévue

Questions de compréhension

1. Pourquoi utilise-t-on X et Z plutôt que X et Y pour le déplacement au sol?

_____

_____

2. Pourquoi limiter la magnitude du vecteur?

_____

_____

3. Quelle responsabilité appartient à Update? À FixedUpdate?

_____

_____

4. Pourquoi MovePosition est-il préférable à une modification directe du Transform?

_____

_____

5. Pourquoi la caméra effectue-t-elle son suivi dans LateUpdate?

_____

_____

6. Pourquoi BatterieCollectable informe-t-il le gestionnaire?

_____

_____

7. Pourquoi le gestionnaire ne devrait-il pas déplacer directement le battant?

_____

_____

8. Quelle condition empêche la touche E d'agir à distance?

_____

_____

Liste de vérification finale

Vérification	Oui	À corriger
Robot identifié par le tag Player	☐	☐
Rigidbody non cinématique et rotation X Z figée	☐	☐
Déplacement sur X et Z	☐	☐
Vitesse diagonale limitée	☐	☐
Rotation vers le mouvement	☐	☐
Caméra suiveuse fluide	☐	☐
Trois batteries collectables	☐	☐
Gestionnaire comptabilisant les batteries	☐	☐
Porte fermée avant l'objectif	☐	☐
Porte ouverte après l'objectif	☐	☐
Zone dangereuse et point de réapparition	☐	☐
Interrupteur utilisable avec E	☐	☐
Défi autonome terminé	☐	☐
Aucune erreur rouge dans la Console	☐	☐
Scène et scripts enregistrés dans Git	☐	☐

Ce que vous devez démontrer

- le déplacement et la rotation;
- la différence entre Update et FixedUpdate;
- la caméra suiveuse;
- la collecte et l'ouverture conditionnelle;
- la réapparition;
- l'interaction de proximité avec E;
- votre contribution personnelle au projet.